

university of groningen / faculty of science and engineering / mathematics and applied mathematics

Randomized approaches to approximating solutions of large kernel systems for Kernel Ridge Regression using the Nyström method

Bachelor's Project Mathematics

April 2026

Student: Y. Tao

First supervisor: Dr. M. Meier

Second supervisor: Prof. J.G. Peypouquet

Abstract

Kernel ridge regression leads to a linear system involving the full kernel matrix K. This matrix has size $n \times n$, so forming and storing it becomes expensive when the number of data points is large. This project studies randomized Nyström approximations, where only $l \ll n$ columns of K are used to build a low-rank approximation. The aim is to compare how the choice of sampled columns and the number of sampled columns affect accuracy, runtime, memory use, conditioning, and solver behaviour. Gittens and Mahoney (2013)

Contents

1 Introduction **3**

2 Kernel Ridge Regression **5**
2.1 Introduction to the regression problem 5
2.1.1 RKHS and KRR 5
2.1.2 Representer form 6
2.1.3 Kernel choices 8
2.2 Singular values and low-rank structure 8
2.3 Runtime and accuracy experiments 9

3 Nyström Approximation **12**
3.1 General Nyström form 12
3.2 Nyström approximation and the KRR system 14
3.3 Column sampling 14
3.4 Randomized Nyström approximation 17
3.4.1 Uniform sampling 18
3.4.2 Leverage score sampling 19

4 Approximate leverage score sampling **20**

1 Introduction

Kernel methods provide a way to capture nonlinear relationships by projecting data onto a higher-dimensional space and describing the data through pairwise similarities. Kernel functions are used to compute inner products in the higher-dimensional space without explicitly mapping the data. This allows flexible regression models to be constructed, but the resulting matrix computations become expensive as the number of observations increases (Alaoui and Mahoney, 2015). This project therefore studies randomized Nyström approximation, one of the most popular techniques that reduces this computational cost for kernel ridge regression while trying to preserve its prediction quality (Musco and Musco, 2017).

Many regression problems start with data (x_i, y_i) for $i = 1, \ldots, n$, where n is the number of observations, $x_i \in \mathbb{R}^d$ are the inputs and $y_i \in \mathbb{R}$ are the outputs. We would like to use these data to create a model for the relationship between the inputs and outputs. One formulation of this task is kernel ridge regression, which uses a kernel function k to measure pairwise similarities between the inputs. These values form the $n \times n$ kernel matrix K, whose entries are $K_{ij} = k(x_i, x_j)$. The details of this formulation are introduced in Chapter 2.

For large datasets, the main computational difficulty is the need to explicitly form the full matrix K, which costs $O(n^2)$ time and memory. This is the part that becomes infeasible first and takes the most time and resources. Solving the resulting system directly can also cost about $O(n^3)$ operations, so this is fine for small tests, but not realistic when n becomes large (Drineas et al., 2005; Alaoui and Mahoney, 2015).

Therefore in this project we investigate the Nyström approximation of the kernel matrix. Instead of forming all of K, we select an index set $I \subset \{1, \ldots, n\}$ containing $l \ll n$ column indices. The selected columns form the matrix $C = K_{:,I} \in \mathbb{R}^{n \times l}$, while their intersections with the corresponding rows form the smaller matrix $W = K_{I,I} \in \mathbb{R}^{l \times l}$. These two matrices are then used to construct a low-rank approximation $\tilde{K} \approx K$, for example $\tilde{K} = CW^{\dagger}C^T$. This is a randomized approach for kernel matrices (Drineas et al., 2005; Gittens and Mahoney, 2013). The point is to reduce the computational cost while still keeping enough information from the original kernel matrix. The full construction is introduced in Chapter 3.

Nyström needs a set of l columns to build $\tilde{K}$. The simplest choice is uniform random sampling, i.e. pick columns uniformly at random. Smarter strategies try to pick columns that are more informative so that fewer columns are needed for similar accuracy. One idea is to sample according to ridge leverage scores, which give higher probability to points that matter more for the kernel ridge regression solution (Alaoui and Mahoney, 2015). Another idea is to approximate these leverage type probabilities adaptively without ever constructing the whole K (Calandriello et al., 2017).

This project is therefore about how to decide which columns to sample and how many columns l are needed for a good Nyström approximation in the KRR setting. We will compare different sampling strategies, starting from uniform random sampling as a baseline, and then compare against smarter strategies motivated by the literature. The experiments will compare these strategies in terms of prediction accuracy and computational complexity, such as runtime and memory, as l changes.

The research questions are:

- What Nyström column sampling strategies give good performance for KRR in terms of prediction accuracy vs runtime/memory?
- How large does l need to be to get a good approximation, and how does this depend

on the sampling strategy and λ?

- What do the accuracy and cost (runtime/memory) trade-offs look like when l increases, and can we recommend a reasonable approach?
- How do conditioning and solver convergence relate to λ and the Nyström approximation quality?

This thesis is organised as follows. Chapter 2 introduces kernel ridge regression and the kernel matrices used in the project. It also gives the first singular value experiments, which motivate why a low-rank approximation is a reasonable thing to try. Chapter 3 introduces the Nyström approximation, starting from the general positive semidefinite matrix formula and then moving to column sampling.

2 Kernel Ridge Regression

In this chapter we introduce the kernel ridge regression problem that lays the foundation for this project. First, the regression problem is written as an optimization problem over a reproducing kernel Hilbert space. Then the representer theorem is used to turn this function-space problem into a finite-dimensional problem involving the kernel matrix. Finally, we introduce the kernels used in the experiments and give some numerical checks that motivate using low-rank approximations.

2.1 Introduction to the regression problem

In this section we introduce the regression problem that will be used throughout the project. Assume we have a dataset (x_i, y_i) for $i = 1, \ldots, n$, where $x_i \in \mathbb{R}^d$ are inputs and $y_i \in \mathbb{R}$ are outputs. We would like to use these data to create a model for the relationship between the inputs and outputs.

In ordinary linear regression, the model is searched for in a finite-dimensional linear function class. Kernel ridge regression instead searches over a reproducing kernel Hilbert space $\mathcal{H}$.

2.1.1 RKHS and KRR

A positive definite kernel k defines a function space starting from finite linear combinations of kernel sections $k(\cdot, x_i)$. More precisely, one first considers functions of the form

$$f(\cdot) = \sum_{i=1}^{n} \alpha_i k(\cdot, x_i).$$

On this space, the inner product is defined as

$$\|f\|_{\mathcal{H}}^2 = \left\langle \sum_{i=1}^{n} \alpha_i k(\cdot, x_i), \sum_{j=1}^{n} \alpha_j k(\cdot, x_j) \right\rangle_{\mathcal{H}},$$

which can be simplied to

$$\langle k(\cdot, x_i), k(\cdot, x_j) \rangle_{\mathcal{H}} = k(x_i, x_j).$$

The reproducing kernel Hilbert space $\mathcal{H}$ is then obtained by completing this space. With this inner product, functions in $\mathcal{H}$ satisfy the reproducing property

$$f(x) = \langle f, k(\cdot, x) \rangle_{\mathcal{H}}.$$

This means that evaluating f at a point can be written as an inner product with the kernel section centered at that point (Zhu, 2023).

The RKHS norm $\|f\|_{\mathcal{H}}$ gives a way to penalize the size of the function in this kernel-defined space. One reason this norm can be interpreted as controlling the variation of f is the Cauchy-Schwarz inequality together with the reproducing property. For two inputs x and x',

$$|f(x) - f(x')| = |\langle f, k(\cdot, x) - k(\cdot, x') \rangle_{\mathcal{H}}| \leq \|f\|_{\mathcal{H}} \|k(\cdot, x) - k(\cdot, x')\|_{\mathcal{H}}.$$

Thus a smaller norm limits how much the function can vary relative to the distance induced by the kernel. This motivates the regularization term $\lambda \|f\|_{\mathcal{H}}^2$ in kernel ridge regression (Zhu, 2023).

Kernel ridge regression searches for a function $f \in \mathcal{H}$ that fits the training data while keeping the RKHS norm small. The optimization problem is

$$\min_{f \in \mathcal{H}} \|f(X) - y\|_2^2 + \lambda \|f\|_{\mathcal{H}}^2,$$

which is also

$$\min_{f \in \mathcal{H}} \sum_{i=1}^{n} (f(x_i) - y_i)^2 + \lambda \|f\|_{\mathcal{H}}^2.$$

Here $\lambda > 0$ is the regularization parameter. The first term measures the error on the training data, while the second term penalizes functions with large RKHS norm. In other words, λ controls the trade off between fitting the data and overfitting.

Now let

$$k : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$$

be a positive definite kernel function. For the training data we define the kernel matrix

$$K \in \mathbb{R}^{n \times n}, \qquad K_{ij} = k(x_i, x_j).$$

The entry K_{ij} measures the similarity between the data points x_i and x_j according to the chosen kernel function. The matrix K induced by k is symmetric positive semidefinite. The kernel trick enters when the function-space problem above is rewritten using only kernel evaluations, which leads to a finite-dimensional formulation involving this matrix K.

2.1.2 Representer form

The optimization problem above is written over the whole function space $\mathcal{H}$, so at first it is not clear how it can be solved computationally. The representer theorem gives the key reduction. For optimization problems in an RKHS where the loss function depends on the values $f(x_i)$ at the training points and the penalty is a monotone function of $\|f\|_{\mathcal{H}}^2$, the solution can be written as a finite linear combination of kernel functions centered at the training points (Zhu, 2023). Such a minimizer has the form

$$f(x) = \sum_{j=1}^{n} \alpha_j k(x, x_j),$$

for some coefficient vector

$$\alpha = (\alpha_1, \ldots, \alpha_n)^T \in \mathbb{R}^n.$$

The reason why this is useful is that we no longer need to search over all functions in $\mathcal{H}$. We only need to determine the n coefficients $\alpha_1, \ldots, \alpha_n$. The representer theorem implies that any part of f orthogonal to the span of $k(\cdot, x_1), \ldots, k(\cdot, x_n)$ does not change the values at the training points, but it can only increase the RKHS norm penalty. Hence the minimizer can be taken inside this finite-dimensional span (Zhu, 2023).

We now rewrite both terms in the KRR objective using α and K. For the training points we have

$$f(x_i) = \sum_{j=1}^{n} \alpha_j k(x_i, x_j),$$

so in vector form

$$f(X) = K\alpha.$$

Therefore the data fitting term becomes

$$\sum_{i=1}^{n} (f(x_i) - y_i)^2 = \|K\alpha - y\|_2^2.$$

The RKHS norm term can also be written using the kernel matrix. Since

$$f = \sum_{i=1}^{n} \alpha_i k(\cdot, x_i),$$

we have

$$\|f\|_{\mathcal{H}}^2 = \left\langle \sum_{i=1}^{n} \alpha_i k(\cdot, x_i), \sum_{j=1}^{n} \alpha_j k(\cdot, x_j) \right\rangle_{\mathcal{H}}.$$

Using bilinearity of the inner product,

$$\|f\|_{\mathcal{H}}^2 = \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j \langle k(\cdot, x_i), k(\cdot, x_j) \rangle_{\mathcal{H}}.$$

By the reproducing property,

$$\langle k(\cdot, x_i), k(\cdot, x_j) \rangle_{\mathcal{H}} = k(x_i, x_j).$$

We then have

$$\|f\|_{\mathcal{H}}^2 = \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j k(x_i, x_j).$$

Since $K_{ij} = k(x_i, x_j)$ this becomes

$$\|f\|_{\mathcal{H}}^2 = \alpha^T K \alpha.$$

Substituting the data fitting term and the RKHS norm term into the objective gives the finite-dimensional problem

$$\min_{\alpha \in \mathbb{R}^n} \|K\alpha - y\|_2^2 + \lambda \alpha^T K \alpha.$$

Expand

$$\|K\alpha - y\|_2^2 = (K\alpha - y)^T (K\alpha - y).$$

Then differentiate

$$(K\alpha - y)^T (K\alpha - y) + \lambda \alpha^T K \alpha$$

with respect to α. Since K is symmetric this gives

$$2K(K\alpha - y) + 2\lambda K\alpha = 0.$$

This then reduces to

$$(K + \lambda I)\alpha = y.$$

Therefore the coefficient vector is

$$\alpha = (K + \lambda I)^{-1} y.$$

The prediction for a new point x is then

$$\hat{f}(x) = \sum_{j=1}^{n} \alpha_j k(x, x_j).$$

If X_{test} is a set of test points and $K_{\text{test,train}}$ is the matrix with entries

$$(K_{\text{test,train}})_{ij} = k(x_i^{\text{test}}, x_j^{\text{train}}),$$

then the test predictions are

$$\hat{y}_{\text{test}} = K_{\text{test,train}}\alpha.$$

The regularization term is important numerically. Since K can have very small eigenvalues or even zero eigenvalues, adding λI shifts all eigenvalues by λ so $K + \lambda I$ is positive definite. This makes the system uniquely solvable and improves the conditioning of the problem.

For large datasets, the main computational difficulty is that K is an $n \times n$ matrix. Forming the full kernel matrix requires $O(n^2)$ kernel evaluations and storing it requires $O(n^2)$ memory. Solving the system directly, for example by Cholesky decomposition, costs $O(n^3)$ operations. For small n this is fine but for large datasets this quickly becomes infeasible.

2.1.3 Kernel choices

In the experiments we start with three common kernels. The linear kernel is

$$k(x, z) = x^T z.$$

The polynomial kernel used in the first tests is

$$k(x, z) = (x^T z + 1)^2.$$

The Gaussian radial basis function kernel (rbf) is

$$k(x, z) = \exp(-\gamma \|x - z\|_2^2).$$

The parameter γ controls how local the Gaussian kernel is (Zhu, 2023; scikit-learn developers, 2026). Larger γ makes points only strongly similar to very nearby points while smaller γ makes the similarities decay more slowly.

These choices give a range of model flexibility. The linear kernel is the simplest option and is used to capture linear structure in the data. In high-dimensional and sparse data, linear kernel has the advantage of avoiding overfitting. The polynomial kernel can capture nonlinear feature interactions unlike the linear kernel. The RBF kernel is widely used for more complex patterns and can represent both linear and nonlinear relationships between data points (GeeksforGeeks, 2025). Since this project is about KRR rather than SVM, the above statements are only used as motivation for comparing the kernels, not as a rule to determine the best kernel.

2.2 Singular values and low-rank structure

Given a large matrix, if many singular values are very small then much of the matrix can be approximated using a smaller rank. This is why in this section we look at singular values of K for real data. The point is to check whether a low-rank approximation is reasonable for real world data.

For this real data experiment we use the Superconductivity dataset from the UCI Machine Learning Repository (Hamidieh, 2018). This is a regression dataset with $n = 21263$ samples and $d = 81$ numerical input features, and the output is the critical temperature. This fits the KRR setting since the target variable is continuous.

This dataset is also large enough to show the computational problem. If the full dataset is used, the kernel matrix has size $n \times n$ where $n = 21,263$, this means there are $n^2 \approx 10^8$ entries in K which corresponds to about 3.6 GB in memory, not to mention the construction computational cost.

This is before solving the linear system or computing any decomposition. So the full matrix is already expensive, and this is the reason that an approximation method such as Nyström is relevant.

We now look at the singular values of three kernels linear, polynomial, and Gaussian RBF. The singular values are computed using the full kernel matrix, so this is just a check of the low-rank structure. The point is to see whether the singular values decay in practice for real data.

For the singular value plots, the singular values are normalised by the largest singular value, so the plotted quantity is

$$\frac{\sigma_i(K)}{\sigma_1(K)}.$$

This makes the three kernels easier to compare since all three curves start at 1. The plots are also stopped once the singular values reach around 10^{-16}, since after that the remaining tail is machine precision.

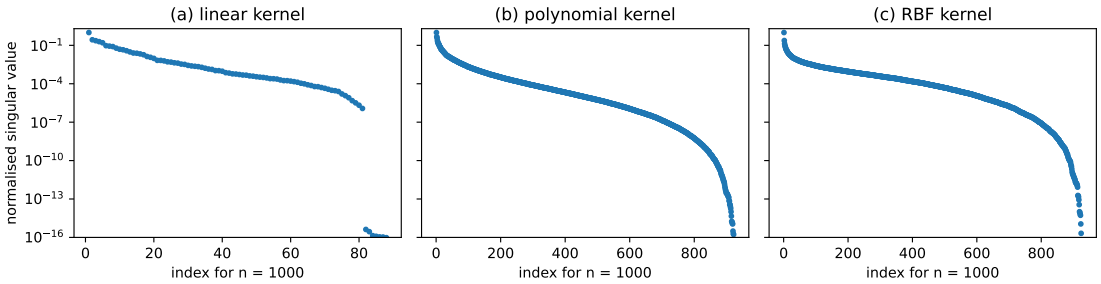


Figure 2.1: Normalised singular value decay for the three kernel matrices: (a) linear, (b) polynomial and (c) RBF.

Figure 2.1 compares the three kernels used in this project. Figure 2.1(a) shows the linear kernel. Since $K = XX^T$ for a linear kernel and the data have 81 features, the rank is at most 81, so the singular values drop to zero after that point. Figure 2.1(b) shows the polynomial kernel. This kernel is still based on inner products, but the square introduces nonlinear factors, so its spectrum does not have the same clean cut-off as the linear kernel. Figure 2.1(c) shows the RBF kernel. Here the decay depends more on the geometry of the data and the value of γ, so this is the most interesting case out of all three kernels.

2.3 Runtime and accuracy experiments

We now check the computational cost in this section. Constructing a full kernel matrix means evaluating pairwise similarities between observations so the expected cost is quadratic in n. Solving the full KRR has cubic asymptotic cost if we use Cholesky or some other direct solver. However in the tested range the construction of K can still take longer than the solve. This does not contradict the asymptotic costs. It just means the $O(n^2)$ construction curve can start much higher since every kernel entry has to be evaluated and stored while the solve is done by highly optimized linear algebra methods. In practice the construction is almost always more expensive than the solve.

Figure 2.2 shows the construction time for the linear, polynomial and RBF kernels. The general behaviour follows the expected quadratic growth, but the constants are not the same. RBF kernel is slower since it is a more complex function than the other two.

Figure 2.3 compares the time to construct the RBF kernel matrix with the time to solve $(K + \lambda I)\alpha = y$. In the code, we measure the solve time by forming $K + \lambda I$, then applying a Cholesky factorization and two triangular solves. The construction time is

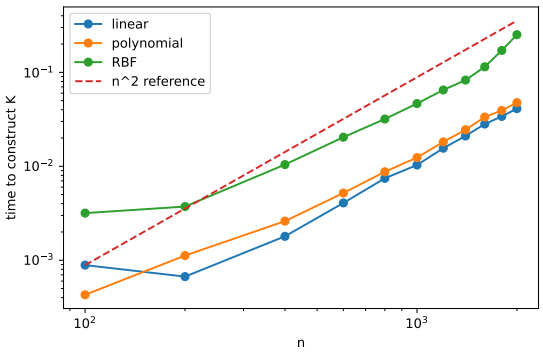


Figure 2.2: Time to construct the full kernel matrix for different values of n.

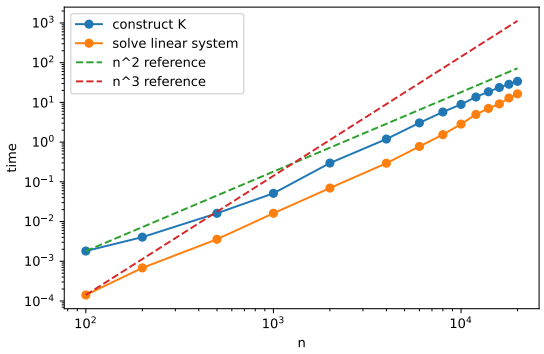


Figure 2.3: Time to construct K vs time to solve the full KRR system.

larger for the increasing values of n which agrees with the theoretical expectation we have discussed above. This is one of the main reasons why column sampling is important: if only l columns are sampled, then we can build $K_{:,I}$ and $K_{I,I}$ instead of all of K which we will see in Chapter 3.

We now look at the behaviour of the full KRR solution as the regularization parameter λ changes. This is important because λ appears directly in the system $(K + \lambda I)\alpha = y$, so changing λ changes both the model and the conditioning of the linear system. Later, when K is replaced by an approximation $\tilde{K}$ in Nyström, the same parameter still appears in the approximation system. Therefore it is useful to first understand the effect of λ in the full KRR system.

For this experiment, we use a fixed train/test split and measure the test error using MSE. We used MSE since it is a natural metric for the regression error and it gives a simple baseline that later can be compared with the Nyström approximations.

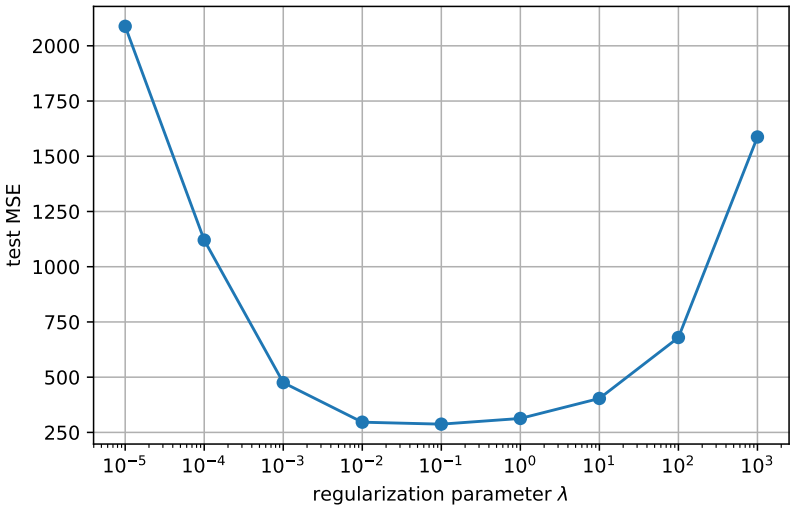


Figure 2.4: Test MSE for full KRR as λ changes.

Figure 2.4 shows that the test MSE depends strongly on λ. For very small values, especially $\lambda = 10^{-5}$ and $\lambda = 10^{-4}$, the test error is high. In this range the regularization term is too weak, so the solution can fit the training data too closely and generalize poorly. As λ increases, the test MSE drops sharply and reaches its lowest value around $\lambda = 10^{-1}$. After this point the error rises again, and for very large values such as $\lambda = 10^2$ and $\lambda = 10^3$,

the penalty term is too strong and the fitted function becomes too restricted. Thus the plot shows the trade-off controlled by λ: both very small or very large λ give less stable fits, small lambda underregularizes which causes overfitting and large lambda overregularizes which causes underfitting. The best value in this small experiment is in the middle of the tested range at roughly $\lambda = 0.1$.

Finally, we fix the test set and use increasingly larger subsets of the training set. This checks both the prediction error and the runtime as the number of training observations grows.

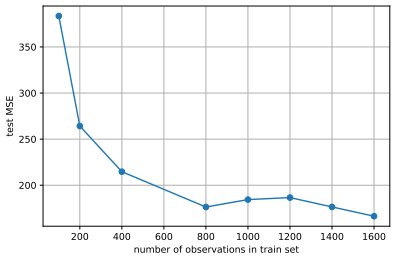


Figure 2.5: Test MSE as the number of training observations increases.

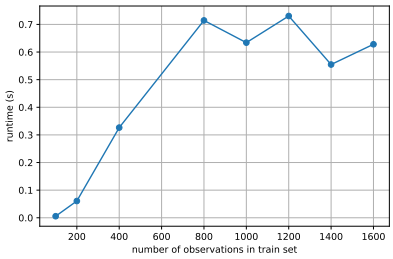


Figure 2.6: Runtime as the number of training observations increases.

Figures 2.5 and 2.6 show the basic trade off. More training observations can improve the test error but the runtime increases at the same time. This is the practical situation where Nyström becomes very useful: ideally we want to keep majority of the predictive information while avoiding the computational cost of dealing with the full kernel matrix.

3 Nyström Approximation

The Nyström approximation is a way to approximate a large positive semidefinite matrix by using only a smaller part of it. In this project the positive semidefinite matrix is the kernel matrix K, but it is useful to first describe the construction for a general positive semidefinite matrix A. The aim is to build a low rank approximation $\tilde{A}$ without using all of A in a full direct factorization.

3.1 General Nyström form

Following the general formulation in randomized numerical linear algebra, let

$$A \in \mathbb{R}^{n\times n}, \quad A \succeq 0,$$

and let $X \in \mathbb{R}^{n\times l}$ be a test matrix, the Nyström approximation of A with respect to X is

$$A\langle X\rangle = (AX)(X^TAX)^\dagger(AX)^T.$$

Here $A \succeq 0$ denotes that A is positive semidefinite, $(X^TAX)^\dagger$ denotes the Moore–Penrose pseudoinverse. The pseudoinverse generalizes the inverse to matrices that may be singular and it agrees with the usual inverse when X^TAX is invertible. The test matrix X is the thing we use to probe or sketch A by forming AX (Martinsson and Tropp, 2020). Later on when we sample columns this matrix becomes a sampling matrix that selects columns of the kernel matrix. The matrix AX contains the information we collect from A. The smaller matrix

$$X^TAX \in \mathbb{R}^{l\times l}$$

describes how the sampled directions interact with each other.

This formula shows why the approximation has rank at most l since the formula is a product involving the $n \times l$ matrix AX, so

$$\operatorname{rank}(A\langle X\rangle) \le l.$$

Therefore, when $l \ll n$, the approximation is much cheaper to store and compute with than the full matrix A.

There is also a useful geometric representation of this formula. Since A is positive semidefinite, it has a positive semidefinite square root $A^{1/2}$ and $A = A^{1/2}A^{1/2}$. Therefore

$$AX = A^{1/2}(A^{1/2}X)$$

and

$$(AX)^T = (A^{1/2}X)^TA^{1/2}.$$

Also,

$$X^TAX = X^TA^{1/2}A^{1/2}X = (A^{1/2}X)^T(A^{1/2}X).$$

Substituting these into the Nyström approximation formula gives

$$A\langle X\rangle = A^{1/2}[(A^{1/2}X)((A^{1/2}X)^T(A^{1/2}X))^\dagger(A^{1/2}X)^T]A^{1/2}.$$

If we set

$$Y = A^{1/2}X.$$

Then

$$Y^TY = (A^{1/2}X)^T(A^{1/2}X) = X^TAX.$$

Then the expression inside the bracket becomes

$$Y(Y^TY)^\dagger Y^T.$$

We see that this is the standard formula for the orthogonal projector onto the column space of Y. Therefore, if we let P_Y denote this projector, then

$$A\langle X\rangle = A^{1/2}P_Y A^{1/2}.$$

Since $Y = A^{1/2}X$, we can also just write this as $P_{A^{1/2}X}$. Hence

$$A\langle X\rangle = A^{1/2}P_{A^{1/2}X}A^{1/2}.$$

This form explains the error formula. Since $A = A^{1/2}IA^{1/2}$, we have

$$A - A\langle X\rangle = A^{1/2}IA^{1/2} - A^{1/2}P_{A^{1/2}X}A^{1/2}.$$

Factoring out $A^{1/2}$ on both sides gives

$$A - A\langle X\rangle = A^{1/2}(I - P_{A^{1/2}X})A^{1/2}.$$

So the error is the part of A that is not captured by the subspace spanned by the columns of $A^{1/2}X$.

This has error as

$$A - A\langle X\rangle = A^{1/2}(I - P_{A^{1/2}X})A^{1/2}$$

which is the Schur complement viewpoint used by Martinsson and Tropp (Martinsson and Tropp, 2020). This means that the Nyström approximation keeps the part of A which is seen through the subspace generated by X. We now give a formal theorem of this.

Theorem 3.1. *Let $A \in \mathbb{R}^{n\times n}$ be positive semidefinite and let $X \in \mathbb{R}^{n\times l}$. Then*

$$A - A\langle X\rangle = A^{1/2}(I - P_{A^{1/2}X})A^{1/2}.$$

In particular,

$$A - A\langle X\rangle \succeq 0,$$

so

$$0 \preceq A\langle X\rangle \preceq A.$$

We have proved parts of this theorem above, to complete it we see that since $I - P_{A^{1/2}X}$ is an orthogonal projector, it is positive semidefinite. Thus

$$A - A\langle X\rangle \succeq 0.$$

This also gives

$$0 \preceq A\langle X\rangle \preceq A.$$

Theorem 3.1 identifies the error matrix of the Nyström approximation. The left hand side $A - A\langle X\rangle$ is exactly the part of A that is not captured by the approximation. The right hand side shows that this error is determined by the complementary projector $I - P_{A^{1/2}X}$. Since $P_{A^{1/2}X}$ projects onto the column space of $A^{1/2}X$, the matrix $I - P_{A^{1/2}X}$ projects onto complementary part of that space in the whole space. Therefore the theorem says that the Nyström error comes from the directions of A that are missed by the subspace generated by $A^{1/2}X$ (Martinsson and Tropp, 2020).

This shows why the choice of X matters. If the columns of $A^{1/2}X$ capture the important directions of A, then the missed part $A^{1/2}(I - P_{A^{1/2}X})A^{1/2}$ should be smaller. If they miss

important directions, then the approximation error can be large. In the column-sampling case, this means that the quality of the Nyström approximation depends on which columns are sampled. This motivates later sections to compare uniform sampling, leverage-score sampling and approximate leverage-score sampling.

The statement $0 \preceq A\langle X\rangle \preceq A$ says that the Nyström approximation does not create a matrix larger than A in the positive semidefinite order. This means that it does not invent or exaggerate in any directions in A and it approximates A from below. In the KRR setting this matters because replacing K by $\tilde{K}$ changes the regularized system from

$$(K + \lambda I)\alpha = y$$

to

$$(\tilde{K} + \lambda I)\tilde{\alpha} = y.$$

Since $\tilde{K} \preceq K$, the approximation changes both the estimator and the conditioning of the linear system. In directions where $\tilde{K}$ approximates K well, the solution should be close. In directions where $\tilde{K}$ loses a lot information, the λI regularization term dominates more. This is one reason why the number of sampled columns l, the sampling strategy and the regularization parameter λ have to be studied together.

3.2 Nyström approximation and the KRR system

We now discuss the practical implementation of the Nyström approximation. Since K is symmetric positive semidefinite, it has an eigendecomposition

$$K = U\Lambda U^T.$$

Splitting the eigenvectors into essential and less important directions gives

$$U = \begin{bmatrix} U_1 & U_2 \end{bmatrix}, \qquad \Lambda = \begin{bmatrix} \Lambda_1 & 0 \\ 0 & \Lambda_2 \end{bmatrix}.$$

Therefore

$$K = U_1\Lambda_1 U_1^T + U_2\Lambda_2 U_2^T.$$

If the eigenvalues in Λ_2 are small, then the second term contributes less to the matrix, and the ideal rank-l approximation is $K \approx U_1\Lambda_1 U_1^T$. This motivates the approximation idea: instead of using all directions of K, we would like to keep only the important directions. We have seen this idea in the section on the general Nyström form, where the approximation error is determined by the directions that are missed by the subspace generated by X. In this case, the important directions are those corresponding to the eigenvectors in U_1.

However, computing U_1 and Λ_1 directly would require spectral information from the full kernel matrix. This is not practical if the goal is to avoid computing with all of K. Thus we use construct a low rank approximation from sampled columns with $\tilde{K} = CW^{\dagger}C^T$. Here $C = K_{:,I}$ contains the sampled columns, and $W = K_{I,I}$ is the smaller intersection matrix. This is not the exact truncated eigendecomposition, but it is a column sampling way to approximate the important low rank part of K as we we will see in the next section (Musco and Musco, 2017).

3.3 Column sampling

We now look at column sampling. Let

$$I = \{i_1, \ldots, i_l\} \subset \{1, \ldots, n\}$$

be the selected column indices. Define the sampling matrix

$$S = \begin{bmatrix} e_{i_1} & e_{i_2} & \cdots & e_{i_l} \end{bmatrix} \in \mathbb{R}^{n \times l},$$

where e_i is the i-th standard basis vector. Taking $X = S$ in the general Nyström formula gives

$$\tilde{K} = K\langle S \rangle = (KS)(S^T KS)^\dagger (KS)^T.$$

Now define

$$C = KS = K_{:,I} \in \mathbb{R}^{n \times l}$$

and

$$W = S^T KS = K_{I,I} \in \mathbb{R}^{l \times l}.$$

Then the Nyström approximation becomes

$$\tilde{K} = CW^\dagger C^T.$$

Equivalently,

$$\tilde{K} = K_{:,I}(K_{I,I})^\dagger K_{I,:}.$$

This is the form used in this project. Instead of forming or storing all of K, the approximation is built from the sampled columns $K_{:,I}$ and the smaller intersection matrix $K_{I,I}$. The notation $K_{:,I}$ means that we keep all rows of K, but only the columns whose indices are in I. In other words,

$$K_{:,I} = \begin{bmatrix} K_{1,i_1} & K_{1,i_2} & \cdots & K_{1,i_l} \\ K_{2,i_1} & K_{2,i_2} & \cdots & K_{2,i_l} \\ \vdots & \vdots & & \vdots \\ K_{n,i_1} & K_{n,i_2} & \cdots & K_{n,i_l} \end{bmatrix}$$

Similarly, $K_{I,I}$ means that both the rows and columns only contain that of the selected index set I:

$$K_{I,I} = \begin{bmatrix} K_{i_1,i_1} & K_{i_1,i_2} & \cdots & K_{i_1,i_l} \\ K_{i_2,i_1} & K_{i_2,i_2} & \cdots & K_{i_2,i_l} \\ \vdots & \vdots & & \vdots \\ K_{i_l,i_1} & K_{i_l,i_2} & \cdots & K_{i_l,i_l} \end{bmatrix}$$

If W is invertible then the formula simplifies to

$$\tilde{K} = CW^{-1}C^T.$$

However in practice $W^\dagger$ is safer to write since W can be singular or nearly singular.

The column-sampling formula above also gives a simple Nyström algorithm. The steps are:

1. choose l landmark indices $I = \{i_1, \ldots, i_l\}$;
2. form matrix $C = K_{:,I}$ and $W = K_{I,I}$;
3. compute the pseudoinverse $W^\dagger$;
4. approximate K by $\tilde{K} = CW^\dagger C^T$.

The approximate KRR system obtained from the Nyström approximation is

$$(CW^{-1}C^T + \lambda I)\tilde{\alpha} = y.$$

If we first form the full matrix $CW^{-1}C^T$, then we again have an $n \times n$ matrix. This is useful in small experiments, but it does not use the full advantage of the Nyström representation. What we can do is to keep C and W, then reduce the solve to a smaller system.

Assume that W is invertible. Let

$$q = W^{-1}C^T\tilde{\alpha}.$$

Then

$$CW^{-1}C^T\tilde{\alpha} = Cq.$$

Therefore the approximate KRR system becomes

$$Cq + \lambda\tilde{\alpha} = y.$$

Solving this equation for $\tilde{\alpha}$ gives

$$\tilde{\alpha} = \frac{1}{\lambda}(y - Cq).$$

Substituting this into the definition of q gives

$$q = W^{-1}C^T\left(\frac{1}{\lambda}(y - Cq)\right).$$

Multiplying by λW gives

$$\lambda Wq = C^T y - C^T Cq.$$

Rearranging gives

$$(C^T C + \lambda W)q = C^T y.$$

This is only a $l \times l$ linear system. After solving for q, the approximate KRR coefficient vector is recovered by

$$\tilde{\alpha} = \frac{1}{\lambda}(y - Cq).$$

So the large system

$$(CW^{-1}C^T + \lambda I)\tilde{\alpha} = y$$

is replaced by the smaller system

$$(C^T C + \lambda W)q = C^T y.$$

The full $n \times n$ matrix $CW^{-1}C^T$ is never formed explicitly. The main matrices used in the solve are $C \in \mathbb{R}^{n\times l}$, $W \in \mathbb{R}^{l\times l}$ and $C^T C \in \mathbb{R}^{l\times l}$.

The computational advantage is therefore as follows. Forming $C^T C$ costs $O(nl^2)$ and solving the smaller system costs $O(l^3)$. When $l \ll n$, this is much cheaper than solving a dense $n \times n$ system, which costs about $O(n^3)$.

If W is singular, the inverse W^{-1} is replaced by the Moore–Penrose pseudoinverse $W^{\dagger}$. We factor $W^{\dagger}$ as $W^{\dagger} = BB^T$ and write

$$CW^{\dagger}C^T = CBB^T C^T = DD^T,$$

where $D = CB$. Then the approximate KRR system becomes

$$(DD^T + \lambda I)\tilde{\alpha} = y.$$

Now let $z = D^T\tilde{\alpha}$, then

$$DD^T\tilde{\alpha} = Dz,$$

so the system becomes

$$Dz + \lambda\tilde{\alpha} = y.$$

Solving for $\tilde{\alpha}$ gives

$$\tilde{\alpha} = \frac{1}{\lambda}(y - Dz).$$

Substituting this into the definition of z gives

$$z = D^T\left(\frac{1}{\lambda}(y - Dz)\right).$$

Multiplying by λ and rearranging gives

$$(D^TD + \lambda I)z = D^Ty.$$

After solving for z, the approximate KRR coefficient vector is recovered by

$$\tilde{\alpha} = \frac{1}{\lambda}(y - Dz).$$

Therefore, in the singular case, the large system

$$(CW^\dagger C^T + \lambda I)\tilde{\alpha} = y$$

is handled through the factorization

$$CW^\dagger C^T = DD^T$$

and reduced to the smaller system

$$(D^TD + \lambda I)z = D^Ty.$$

The full $n \times n$ matrix $CW^\dagger C^T$ is also never formed explicitly. The additional work is the factorization of $W^\dagger$, which only involves the small $l \times l$ matrix W.

3.4 Randomized Nyström approximation

The randomized Nyström approximation draws a random matrix

$$\Omega \in \mathbb{R}^{n\times l}$$

and form

$$Y = A\Omega.$$

Then the Nyström approximation is

$$\hat{A} = Y(\Omega^TY)^\dagger Y^T.$$

This is the randomized Nyström approximation described by Martinsson and Tropp (Martinsson and Tropp, 2020). The idea is that the random matrix Ω collects information about the important directions of A. If the column space of $A\Omega$ captures the dominant eigenspace of A well then the approximation error should be small.

For kernel matrices the usual Nyström method samples columns of K. This corresponds to taking $\Omega = S$, where S is a column selection matrix. Then

$$Y = KS = K_{:,I},$$

and the formula becomes

$$\tilde{K} = K_{:,I}(K_{I,I})^\dagger K_{I,:}.$$

Thus in this project randomness enters through the random choice of the index set I. Different choices of the distribution for I give different Nyström methods.

3.4.1 Uniform sampling

The baseline method is uniform sampling. This means choosing l column indices from $\{1, \ldots, n\}$ uniformly at random. It is easy to implement and cheap because no importance scores have to be computed. Its weakness is that it treats all columns equally. If some columns are much more informative than others, uniform sampling may need a larger l to reach the same accuracy as a smarter strategy.

Before stating the theorem for this, we need the idea of coherence. Let K be a positive semidefinite kernel matrix and let $K = U\Lambda U^T$ be its eigendecomposition. Let $U_k \in \mathbb{R}^{n\times k}$ contain the leading k eigenvectors. The leverage score of row or column i relative to this k-dimensional eigenspace is $s_i = \|U_k(i,:)\|_2^2$. These scores satisfy $\sum_{i=1}^n s_i = k$.

This measures how concentrated the leading eigenspace is in a few rows. If the scores are all similar, then μ is not large and uniform sampling has a better chance of working. If a few scores are much larger than the rest, then uniform sampling can easily miss important columns.

This issue is discussed by Gittens and Mahoney. In some datasets the leverage scores are fairly uniform so uniform sampling can be reasonable. In other datasets the leverage scores are highly nonuniform and uniform sampling can be much worse (Gittens and Mahoney, 2013). This is why uniform sampling is a natural baseline method but not necessarily the best method.

Theorem 3.2 (Uniform sampling and coherence)**.** *Let $K \in \mathbb{R}^{n\times n}$ be positive semidefinite, and let K_k be its best rank k approximation. Fix a failure probability δ and an accuracy factor ϵ both in $(0,1)$. Suppose l columns are sampled uniformly at random. In the basic Nyström method we are using here, this corresponds to the $q = 1$ case of Lemma 8 in Gittens and Mahoney, so*

$$C = KS = K_{:,I}, \qquad W = S^T KS = K_{I,I}.$$

If

$$l \geq 2\mu\epsilon^{-2}k\ln(k/\delta),$$

then, with probability at least $1-3\delta$, the Nyström approximation $\tilde{K} = CW^{\dagger}C^T$ satisfies the spectral norm bound

$$\|K - \tilde{K}\|_2 \leq \left(1 + \frac{n}{(1-\epsilon)l}\right)\|K - K_k\|_2.$$

This bound for l fits the intuition as it shows us when μ is large we need to sample more columns. This is exactly the point that uniform sampling does not know which columns are important so when the coherence is large it needs more columns to capture the essential structure.

In the first experiments we test uniform sampling. For each l, the approximation quality can be measured directly by a relative forward error, for example

$$\frac{\|K - \tilde{K}\|_F}{\|K\|_F}.$$

It is also important to measure the effect on prediction. So the approximation is also judged by what happens when $\tilde{K}$ is used in place of K in the regularized system, this is done by solving the system with $\tilde{K}$ and then measuring the test error of the result.

3.4.2 Leverage score sampling

Uniform sampling is cheap but it ignores the structure of K. Leverage score sampling is the opposite extreme, it tries to sample columns according to how important they are in the leading eigenspace.

The full leverage score version is the most informed method. It uses all of K first:

1. construct the full kernel matrix K;
2. compute the leading eigenvectors U_k;
3. compute the squared row norms $s_i = \|U_k(i,:)\|_2^2$;
4. compute probabilities $p_i = \frac{s_i}{\sum_j s_j}$;
5. sample columns according to these probabilities.

This method is expensive because it requires the full matrix and the leading eigenvectors. But it is still useful as a reference method. It says what the informed sampling distribution would be if the important eigenspace was already known.

Theorem 3.3 (Leverage score sampling)**.** *Let $K \in \mathbb{R}^{n \times n}$ be positive semidefinite and let U_k contain the leading k eigenvectors. Suppose columns are sampled with probabilities satisfying*

$$p_i \geq \frac{\beta}{k} \|U_k(i,:)\|_2^2$$

for some $\beta \in (0,1]$. If the number of sampled columns has order

$$l = O\left(\frac{k \log(k/\beta)}{\beta \epsilon^2}\right),$$

then the Nyström approximation satisfies

$$\|K - \tilde{K}\|_2 \leq \|K - K_k\|_2 + \epsilon^2 \|K - K_k\|_*,$$

where ϵ is a fixed approximation factor.

The important thing here is the dependence on l. Uniform sampling has a coherence factor while leverage score sampling samples according to the row norm scores. This can cause leverage score sampling to use much fewer columns when the leverage scores are very uneven.

4 Approximate leverage score sampling

The full leverage score method is too expensive if the whole reason for Nyström is to avoid working with all of K. The middle method is therefore to look at a small part of K first and use it to estimate which rows or columns look important.

Here is a simple version of this idea:

1. sample 5 columns uniformly;
2. form the small sampled matrix $K_s \in \mathbb{R}^{n \times 5}$;
3. compute an orthonormal basis U_s for the column space of K_s;
4. compute approximate scores as before.
5. sample 95 more columns (total 100 columns) according to these approximate scores.

Bibliography

Alaoui, A. and Mahoney, M. W. (2015). Fast randomized kernel ridge regression with statistical guarantees. *Advances in neural information processing systems*, 28.

Calandriello, D., Lazaric, A., and Valko, M. (2017). Distributed adaptive sampling for kernel matrix approximation. In *Artificial Intelligence and Statistics*, pages 1421–1429. PMLR.

Drineas, P., Mahoney, M. W., and Cristianini, N. (2005). On the nyström method for approximating a gram matrix for improved kernel-based learning. *journal of machine learning research*, 6(12).

GeeksforGeeks (2025). How to choose the best kernel function for svms. GeeksforGeeks.

Gittens, A. and Mahoney, M. (2013). Revisiting the nystrom method for improved large-scale machine learning. In *International Conference on Machine Learning*, pages 567–575. PMLR.

Hamidieh, K. (2018). Superconductivty Data. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C53P47.

Martinsson, P.-G. and Tropp, J. A. (2020). Randomized numerical linear algebra: Foundations and algorithms. *Acta Numerica*, 29:403–572.

Musco, C. and Musco, C. (2017). Recursive sampling for the nystrom method. *Advances in neural information processing systems*, 30.

scikit-learn developers (2026). skylearn.metrics. scikit-learn API.

Zhu, R. (2023). Reproducing kernel hilbert space. Chapter 15 of *Statistical Machine Learning with R*.